Divide & Conquer Strategy & Recurrence Analysis

Comprehensive Exam Study Guide | Algorithm Analysis & Design

1. Asymptotic Notations & Analysis

Asymptotic analysis evaluates the running time or efficiency of an algorithm as the input size **n** approaches infinity (n → &infty;). Rather than measuring execution time in seconds (which varies by hardware and execution environment), it analyzes the growth rate of the time complexity function *T(n)*.

Core Principles:

• **Dominant Term Principle:** For large *n*, lower-order terms and constant coefficients become negligible. For example, in $T(n) = 2n^5 + 10n + 207$, the highest-degree term $2n^5$ dominates, giving an asymptotic complexity of $O(n^5)$.

• **Polynomial Rule:** If *T(n)* is a polynomial of degree *d*, then $T(n) = O(n^d)$.

Notation	Name	Mathematical Condition	Limit Definition	Interpretation
O(g(n))	Big Oh	0 ≤ f(n) ≤ c · g(n)	lim (f(n)/g(n)) < &infty;	Upper bound (Worst-case)
Ω(g(n))	Big Omega	0 ≤ c · g(n) ≤ f(n)	lim (f(n)/g(n)) > 0	Lower bound (Best-case)
Θ(g(n))	Big Theta	0 ≤ c1·g(n) ≤ f(n) ≤ c2·g(n)	0 < lim (f(n)/g(n)) < &infty;	Tight bound (Exact order)
o(g(n))	Little oh	0 ≤ f(n) < c · g(n) (∀c>0)	lim (f(n)/g(n)) = 0	Strict upper bound
ω(g(n))	Little omega	0 ≤ c · g(n) < f(n) (∀c>0)	lim (f(n)/g(n)) = &infty;	Strict lower bound

> **Important Equivalence**
> **Equivalence Relation:** $f(n) = \Theta(g(n)) \Leftrightarrow f(n) = O(g(n))$ *AND* $f(n) = \Omega(g(n))$.

Hierarchy of Common Growth Rates (Fastest to Slowest):

> **Growth Hierarchy**
> $O(1) < O(\log \log n) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(n^k) < O(2^n) < O(n!)$

Formal Constant Proofs (Finding c and n_0):

• **Prove f(n) = 5n + 3 = O(n):** Target 5n + 3 ≤ c·n. For n ≥ 1, 3 ≤ 3n, so 5n + 3 ≤ 5n + 3n = 8n. Choose **c = 8, n_0 = 1**.

• **Prove f(n) = $5n^2$ + 4 = $O(n^2)$:** Target $5n^2 + 4 \le c \cdot n^2$. For n ≥ 2, $4 \le n^2$, so $5n^2 + 4 \le 5n^2 + n^2 = 6n^2$. Choose **c = 6, n_0 = 2**.

• **Prove f(n) = 10n log n + n = O(n log n):** For n ≥ 2, n ≤ n log n. Thus 10n log n + n ≤ 10n log n + n log n = 11n log n. Choose **c = 11, n_0 = 2**.

2. Divide and Conquer Strategy

The **Divide and Conquer** paradigm solves a problem of size *n* by dividing it into *k* distinct subproblems (1 < k < n), solving each recursively, and combining the subproblem solutions.

1. Divide: Partition original problem P into smaller instances P_1, P_2, ..., P_k.

2. Conquer: Recursively solve each subproblem. If subproblem sizes are small enough, solve directly.

3. Combine: Merge solutions of subproblems into the complete solution for P.

> **Control Abstraction Pseudocode**
>
> **Algorithm DAndC(P)**
> **{**
> **if** Small(P) **then return** S(P);
> **else {**
> divide P into smaller instances P_1, P_2, ..., P_k, $k \geq 1$;
> Apply DAndC to each of these subproblems;
> **return** Combine(DAndC(P_1), DAndC(P_2), ..., DAndC(P_k));
> **}**
> **}**

General Divide-and-Conquer Recurrence: $T(n) = a \cdot T(n/b) + f(n)$
Where **a** ≥ **1** is the number of subproblems, **b > 1** is the factor by which input size is divided, and **f(n)** is the cost of division and combination.

3. Methods for Solving Recurrence Relations

Method A: Iteration / Substitution Method (Expansion)

Repeatedly expand the recurrence function into previous terms until a clear pattern emerges, then substitute base case values.

Example 1: T(n) = T(n/2) + 1
Expansion: $T(n) = T(n/4) + 1 + 1 = T(n/8) + 3 = ... = T(n/2^k) + k$.
Base Case: Let $n/2^k = 1 \Rightarrow k = \log_2 n$.
Result: $T(n) = T(1) + \log_2 n =$ **$O(\log n)$**.

Example 2: T(n) = T(n-1) + n
Expansion: $T(n) = T(n-2) + (n-1) + n = T(0) + 1 + 2 + ... + n$.
Summation: $T(n) = T(0) + n(n+1)/2 =$ **$O(n^2)$**.

Example 3: T(n) = 2T(n/2) + cn
Expansion: $T(n) = 2[2T(n/4) + c(n/2)] + cn = 4T(n/4) + 2cn = ... = 2^kT(n/2^k) + k \cdot cn$.
Base Case: Let $n/2^k = 1 \Rightarrow k = \log_2 n$.
Result: $T(n) = n \cdot T(1) + cn \log_2 n =$ **$O(n \log n)$**.

Method B: Substitution Method (Mathematical Induction)

Consists of two steps: (1) **Guess** the form of the solution, and (2) Use **mathematical induction** to prove the constants.
Example: Solve T(n) = 2T(n/2) + n
• **Guess:** $T(n) \leq c \cdot n \log n$.
• **Inductive Step:** Assume $T(n/2) \leq c(n/2) \log(n/2)$.
• **Substitute:** $T(n) \leq 2[c(n/2) \log(n/2)] + n = cn(\log n - \log 2) + n = cn \log n - cn + n$.
• For $c \geq 1$, $-cn + n \leq 0 \Rightarrow T(n) \leq cn \log n \Rightarrow$ **$O(n \log n)$**.

> **Variable Substitution Technique**
>
> **Special Algebraic Technique — Changing Variables:**
> To solve $T(n) = 2T(\sqrt{n}) + \log n$, let $m = \log n \Rightarrow n = 2^m$.
> Substitute: $T(2^m) = 2T(2^{m/2}) + m$. Let $S(m) = T(2^m) \Rightarrow S(m) = 2S(m/2) + m = O(m \log m)$.
> Substitute back $m = \log n \Rightarrow T(n) =$ **$O(\log n \cdot \log \log n)$**.

Method C: Recursion Tree Method

Each node represents the cost of a single subproblem. Sum costs within each level, then sum across all levels.
• **$T(n) = 3T(n/4) + cn^2$:**
  - Level 0 (Root): cn^2
  - Level 1: 3 nodes of size $n/4 \Rightarrow 3 \cdot c(n/4)^2 = (3/16)cn^2$
  - Level i: 3^i nodes of size $n/4^i \Rightarrow (3/16)^i cn^2$
  - Tree Height: $\log_4 n$. Leaf count $= 3^{\log_4 n} = n^{\log_4 3}$.
  - Sum: Geometric series with ratio $3/16 < 1 \Rightarrow$ Sum $\leq (16/13)cn^2 =$ **$O(n^2)$**.
• **$T(n) = T(n/3) + T(2n/3) + cn$:**
  - Cost per level = cn. Longest path height = $\log_{3/2} n$.
  - Total Cost = $cn \cdot \log_{3/2} n =$ **$O(n \log n)$**.

4. Classic Divide and Conquer Algorithms

A. Merge Sort

Splits array into two equal halves, recursively sorts each half, and merges sorted sub-arrays.
• **Recurrence:** $T(n) = 2T(n/2) + \Theta(n)$
• **Time Complexity: O(n log n)** in Worst, Average, and Best cases.
• **Space Complexity:** O(n) auxiliary space for temporary array merging.

> **Merge Function Logic**
>
> **Algorithm Merge(low, mid, high)**
> h := low; i := low; j := mid + 1;
> **while** (h ≤ mid and j ≤ high) **do** {
> **if** (a[h] ≤ a[j]) **then** { b[i] := a[h]; h := h + 1; }
> **else** { b[i] := a[j]; j := j + 1; }
> i := i + 1;
> }
> **if** (h > mid) **then for** k := j **to** high **do** { b[i] := a[k]; i := i + 1; }
> **else for** k := h **to** mid **do** { b[i] := a[k]; i := i + 1; }

B. Quick Sort

Rearranges array elements around a **pivot** element such that elements before pivot are ≤ pivot and elements after pivot are ≥ pivot. In-place sorting algorithm.

- **Worst Case: $O(n^2)$** — occurs when array is already sorted or pivot produces skewed partitions (0 and n-1 elements): $T(n) = T(n-1) + \Theta(n)$.
- **Average Case: O(n log n)** — equal probability for any element to be pivot: $C_A(n) = O(n \log n)$.
- **Best Case: O(n log n)** — balanced split every step: $T(n) = 2T(n/2) + \Theta(n)$.

C. Matrix Multiplication

Multiplying two n × n matrices:

1. **Standard Divide & Conquer:** Divides matrices into 4 submatrices of size (n/2)×(n/2). Requires 8 multiplications and 4 additions.
   Recurrence: $T(n) = 8T(n/2) + O(n^2) \Rightarrow$ **$O(n^3)$**.
2. **Strassen's Algorithm:** Reduces scalar multiplications from 8 to 7 using 18 matrix addition/subtraction operations.
   Recurrence: $T(n) = 7T(n/2) + O(n^2) \Rightarrow T(n) = O(n^{\log_2 7}) \approx$ **$O(n^{2.81})$**.

5. Quick Reference Exam Complexity Table

Algorithm / Recurrence Relation	Time Complexity	Key Properties / Notes
Binary Search	**O(log n)**	Tree height h = log2 n
Merge Sort	**O(n log n)**	T(n) = 2T(n/2) + cn (All cases)
Quick Sort (Best / Average)	**O(n log n)**	Balanced pivot split
Quick Sort (Worst Case)	**O(n2)**	Skewed partition T(n) = T(n-1) + n
Standard Matrix Multiplication	**O(n3)**	T(n) = 8T(n/2) + cn2
Strassen's Matrix Multiplication	**O(n2.81)**	T(n) = 7T(n/2) + an2
T(n) = T(n-1) + 1	**O(n)**	Linear iteration / reduction
T(n) = T(n/2) + 1	**O(log n)**	Halving input size at each step
T(n) = 3T(n/4) + cn2	**O(n2)**	Geometric series bounded by root cost
T(n) = 2T(√n) + log n	**O(log n log log n)**	Variable substitution m = log n